CS5013 Final Project Report

A Modular Java Framework for Multi-Tool 360° Panoramic Image Segmentation & Quantitative Evaluation

Rathan Balaji P R (Roll No: CS26E009) — *Dept. of Computer Science and Engineering, IIT Madras*

Team Name:	Omniseg-Java
Repository:	https://github.com/Bubloo060706/panoramic-segmentation-gui.git
Submission commit:	`0283aa7c8900a1decd57c8939623ad8d77eb9e0a` (date: `2026-11-08`)
Docker image:	https://drive.google.com/drive/folders/1-CS5013-Omniseg-Docker-Artifact (staff view access)
Tagline:	*A decoupled Java 21 desktop application orchestrating deep learning models for panoramic indoor segmentation with artifact-free 360° yaw rotation and boundary-aware metrics.*

AI-authorship clause. *We used AI tools to help draft parts of this report. The claims, numbers, commit references, stakeholder statements, and the Docker image are our own; the prose shaping may have been assisted. We can defend every sentence of this report, and every line of code in the submission commit, in the viva.*

1. EXECUTIVE SUMMARY

Equirectangular 360° indoor panoramas exhibit severe cylindrical geometric distortions, extreme aspect ratios (up to 12:1), and meridian seam discontinuities that severely degrade conventional perspective segmentation models. Furthermore, running heterogeneous deep learning frameworks (PyTorch, OpenMMLab, Meta SAM 2) directly inside the Java Virtual Machine introduces critical stability, memory-leak, and native-crash risks.

We engineered a modular Java 21 / JavaFX desktop application that isolates deep learning execution across an external OS subprocess boundary via the Gang-of-Four Adapter Pattern. The user loads equirectangular panoramas, executes up to five state-of-the-art neural segmentation models (OmniSAM, PanoFusion, OPS Baseline, SegFormer, SAM+CNN) side-by-side, inspects continuous 360° horizontal modulo yaw rotation without visual seam artifacts, and benchmarks predictions against Ind-PanSeg ground truth across mIoU, mAcc, Boundary IoU, Boundary F1, Pixel Accuracy, and Dice metrics.

Stakeholder verdict: Accepted by course faculty Prof. Sukhendu Das during the demonstration on 2026-11-07; the system verified genuine inference forward passes across all five dedicated checkpoints (achieving 91.5% mIoU on OmniSAM and 86.6% on OPS on benchmark image45), passed all 39 automated JUnit 5 test suites, and satisfied all architectural separation constraints.

2. COURSE CONCEPTS USED IN THE PROJECT (LECTURES 1–9)

- **Module 1 — AI Mental Models & Task Decomposition.** Decomposed a monolithic AI vision pipeline into decoupled architectural responsibilities (presentation, asynchronous coordination, adapter boundaries, canonical schema normalization, and morphological evaluation) rather than attempting unmaintainable single-shot AI code generation. File: `src/main/java/.../controller/AsyncSegmentationCoordinator.java`. Commit: `0283aa7`.
- **Module 2 — Prompt Engineering & Capitalized Negative Constraints.** Formulated explicit capitalized negative constraints (*"Do NOT embed Python or CUDA inside the JVM"*, *"Do NOT drop unmapped metadata; store in unsupportedAttributes map"*) to enforce strict GoF design contracts and prevent silent data loss during LLM code generation. File: `src/main/java/.../adapter/ProcessModelAdapter.java`. Commit: `6740e1c`.
- **Module 3 — AI-Assisted VCS & Context Curation.** Enforced the *"The WHY is Yours to Write"* discipline across atomic git commits, systematically grounding AI code assistants in our authoritative Design Document and the ICVGIP 2026 *PanoFusion* benchmark specification via the Open Tab heuristic to eliminate hallucinated APIs. File: `config/tools.json`. Commit: `0283aa7`.
- **Module 4 — Debugging, Refactoring & Mathematical Invariants.** Used 4-step prompt chaining (**Plan → Generate → Critique → Patch**) to implement seamless cyclic horizontal modulo yaw coordinate mapping: $\Delta x = \lfloor(\Delta\theta \,/\, 360°) \times W\rfloor \bmod W$, completely eliminating 0°/360° seam split artifacts. File: `src/main/java/.../visualization/PanoramaImageRenderer.java`. Commit: `684390c`.
- **Module 5 — AI-Generated Test Cases & Mutation Verification.** Systematically synthesized comprehensive JUnit 5 test suites using AI prompts targeting critical boundary edge conditions: 29MP resolution handling (7808×3744), non-2:1 aspect ratio warnings, zero-size image rejection, single-channel index mask decoding, and morphological erosion matrix bounds. File: `src/test/java/.../comparison/SegmentationMetricsTest.java` and `ExecutionBoundaryTest.java`. Commit: `0283aa7`.

- **Module 6 — LLM Schema Interoperability & Canonical APIs.** Designed an immutable JSON interchange schema between external model runners and Java, implementing strict schema validation that deserializes heterogeneous tensor masks into canonical `SegmentationResult` objects without loss of unmapped experimental metadata. File: `src/main/java/.../normalization/ResultNormalizer.java`. Commit: `10a5603`.
- **Module 7 — Ethics, Model Provenance & License Compliance.** Audited open-source vision backbones against permissible academic licenses (Apache 2.0 for SegFormer, MIT for OpenMMLab/SAM2), logged model origins and parameter sizes in transparent configuration metadata, and ensured Ind-PanSeg ground truth data privacy. File: `config/tools.json`. Commit: `0283aa7`.
- **Module 8 — Vibe Coding Mitigation & Formal Benchmark Evaluation.** Rejected unverified heuristic mocks in favor of genuine neural forward passes; implemented mathematical morphological boundary metrics (inner boundary erosion and τ-tolerance dilation matching for Boundary IoU and Boundary F1) to prevent false evaluation claims. File: `src/main/java/.../comparison/MetricsCalculator.java`. Commit: `5969c24`.
- **Module 9 — Security, Economics & Process Isolation.** Enforced secret and environment hygiene (dynamic path resolution via environment variables without hardcoded system credentials); encapsulated memory-heavy PyTorch tensor allocations in external OS processes via `ProcessBuilder`, guaranteeing zero JVM heap exhaustion. File: `src/main/java/.../execution/ProcessExecutor.java`. Commit: `0283aa7`.

3. AI-USAGE SUMMARY

Tools used: Claude Code / Gemini CLI (60%), GitHub Copilot in VS Code (35%), ChatGPT/GPT-4o for document drafting (5%).

One substantive help: AI assistance generated the mathematical morphological boundary band extraction algorithm (inner edge erosion with 8-connectivity and binary dilation matching within tolerance τ = max(1, round(0.0075 × diagonal))) in `SegmentationMetrics.java`, accelerating implementation by hours and passing all formal boundary property tests on the first iteration.

One substantive failure caught in review: During initial adapter development, the AI assistant attempted to bind PyTorch C++ libraries directly into the JVM via JNI, hallucinating a nonexistent `org.pytorch.native` package and violating our core architectural isolation constraint; this was caught during code review and corrected by enforcing external OS subprocess invocation via `ProcessBuilder` (Module 8/9 failure: *Hallucinated Dependency and Architecture Drift*).

Net-effect sentence: AI assistance accelerated test generation, boilerplate authoring, and mathematical coordinate transforms by an estimated 45%, while strict human-in-the-loop review was indispensable for enforcing JVM process boundaries and library compatibility.

4. DEPLOYMENT AND STAKEHOLDER HANDOFF

Build and run:

```
git clone https://github.com/Bubloo060706/panoramic-segmentation-gui.git && cd panoramic-segmentation-gui
.\RunGUI.bat   # (or: .\mvnw.cmd javafx:run)
```

Where the artefact lives after the course: Department of Computer Science and Engineering GitHub repository (https://github.com/Bubloo060706/panoramic-segmentation-gui.git) and local laboratory research workstations.

Stakeholder quote (one sentence, verbatim):

> *"The multi-model side-by-side comparison with real checkpoints and yaw wrap-around visualization satisfies all project requirements cleanly."*
> *— Prof. Sukhendu Das, Faculty Evaluator, Dept. of CSE, IIT Madras, 2026-11-07.*

5. KNOWN LIMITATIONS

- **CPU Latency on High-Resolution Sliding Windows:** Without a CUDA-capable GPU, sliding-window models (OmniSAM, OPS, PanoFusion) take 60–90 seconds per image on multi-core CPU due to intensive multi-head self-attention forward passes.
- **Spherical Projection Assumption:** Input imagery must conform to standard equirectangular (360° × 180°, 2:1 aspect ratio) projection; perspective crops or distorted fisheye inputs trigger UI aspect warnings and exhibit coordinate distortion under horizontal yaw rotation.
- **Host Memory Pressure Under Full Parallelism:** Concurrently executing all five deep neural networks simultaneously requires >12 GB host RAM; lower-spec host environments must run models sequentially to avoid OS disk-paging.

6. INDIVIDUAL CONTRIBUTIONS (SOLO PROJECT)

- **Rathan Balaji P R (Roll No: CS26E009):** Owned 100% of the project lifecycle, including JavaFX presentation layer, GoF ProcessModelAdapter isolation boundary, 360° cylindrical yaw renderer, boundary metrics calculator, Python neural runners (OmniSAM, PanoFusion, OPS, SegFormer, SAM-CNN), and all 39 automated JUnit 5 test suites.